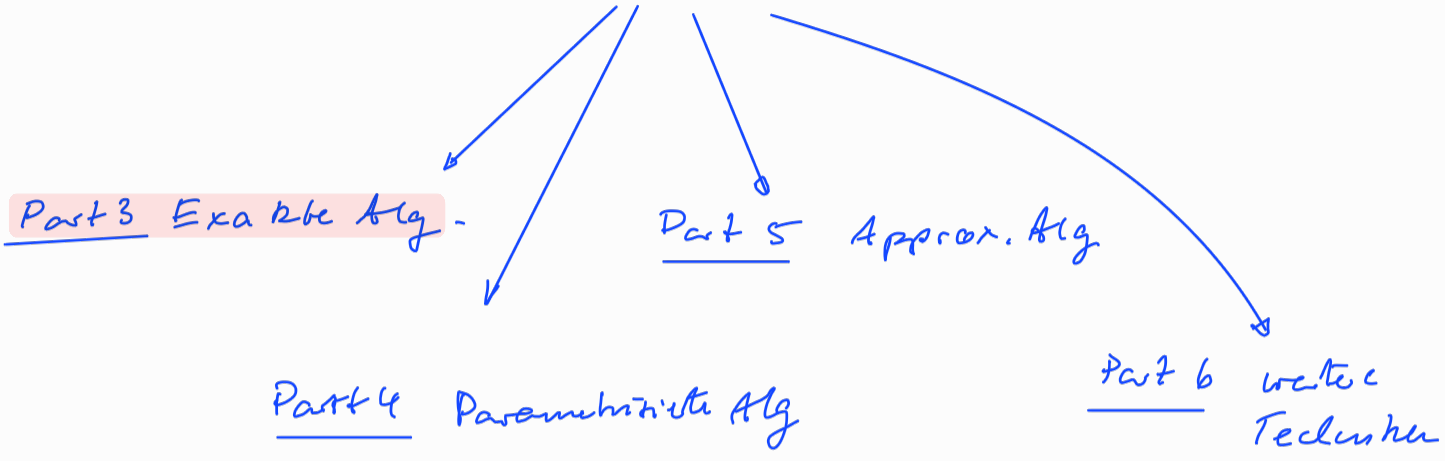
Fahrplan

3.1

Part 1 TM, Algorithmen & Entscheidbarkeit

Part 2 Einfach vs schwere Probleme & NP vollständigkeit



3. Exakte Alg

- akzeptiert exponent. Laufzeit,
  d.h. aber nicht einfach = nur = brute Force!
  also Frage: wenn expon. Laufzeit unvermeidlich,
  wie schlimm wird es dann überlappt?

$n = 50 \rightsquigarrow$ $2^{50} \approx 1{,}1 \cdot 10^{15}$

$1{,}5^{50} \approx 6.4 \cdot 10^{8}$

$1{,}2^{50} \approx 9100$

es kommt auf die Basis aber auch Expon. an!

im folgenden wird für $G = (V, E)$ i.d.R $n := |V|$
& $m := |E|$ sein.

3.1 Divide & Conquer (D&C)

Strategie D&C

Wir zerlegen die Eingabe in 2 mathe. Hälften & kombinieren danach Teilergebnisse.

Schubfach kommt als Technik für Beweis von poly-zeit Alg.

Divide → Teile in kleine disjunkte Teilprobleme
Conquer → löse Teilprobleme
dann kombiniere Teillsg zu Gesamtlsg.

Am Beispiel:

Subset-Sum

Input: $a_1 \dots a_n, T \in \mathbb{N}$

Frage: Gibt es $S \subseteq \{1 \dots n\}$ od $\sum_{i \in S} a_i = T$?

Bsp: $\overset{a_1\ a_2\ a_3\ a_4}{A = \{3, 5, 6, 8\}}$, $T = 14$

hier $S = \{3, 4\}$ gibt Ja-antwort $a_3 + a_4 = 6 + 8 = 14 = T$.

Thm: SUBSET_SUM ist NP-vollständig
└ (ohne Beweis – ÜA).

könnten jetzt alle 2^n Teilmengen $S \subseteq \{1 \dots n\}$ Testen ob $\sum_{i \in S} a_i = T$ ⇒ "schlecht"
wie geht es evtl. besser?

klassischer Weg: "meet in the middle"

SUBSET SUM – DC $(a_1 \dots a_n, T)$

$I_1 = \{1 \dots \lfloor \frac{n}{2} \rfloor\}$

$I_2 = \{\lfloor \frac{n}{2} \rfloor + 1 \dots n\}$ ($\Rightarrow I_1 \dot\cup I_2 = \{1 \dots n\}$, $|I_1| = \lfloor \frac{n}{2} \rfloor$, $|I_2| = \lceil \frac{n}{2} \rceil$)

$L_1 := \{ \sum_{i \in S_1} a_i : S \subseteq I_1 \} \cup \{0\}$

$L_2 := \{ \sum_{i \in S_2} a_i : S_2 \subseteq I_2 \}$

IF ($\exists x \in L_1, y \in L_2$ s.d. $x + y = T$)

THEN return JA

ELSE return NEIN.

(L_1 enthält 0, falls Lsg $y = T$ womit in L_2 liegt. ($x = 0$))

Thm: SUBSET-SUM-DC$(a_1 \dots a_n, T)$ returns JA

$\Leftrightarrow (a_1 \dots a_n, T)$ ist JA-Instanz von SUBSET-SUM

Beweis „$\Rightarrow$" klar, wenn $x \in L_1$ & $y \in L_2$

s.d. $x + y = T$

$\Rightarrow \sum_{i \in S_1} a_i + \sum_{i \in S_2} a_i = T$

da $S_1 \cap S_2 = \emptyset \Rightarrow$ für $S = S_1 \cup S_2$

gilt $S \subseteq I$ & $\sum_{i \in S} a_i = T$

„$\Leftarrow$" Ann $(a_1 \dots a_n, T)$ ist JA-Instanz

$\Rightarrow \exists S \subseteq \{1 \dots n\}$ s.d. $\sum_{i \in S} a_i = T$

klar jedes $i \in S$ liegt entweder in I_1 oder I_2

$\Rightarrow \exists S_1 \subseteq I_1$ & $S_2 \subseteq I_2$ s.d. $S_1 \cap S_2 = \emptyset$ & $S_1 \cup S_2 = S$

$\Rightarrow$ für $x := \sum_{i \in S_1} a_i$ & $y = \sum_{i \in S_2} a_i$

gilt $x + y = \sum_{i \in S} a_i = T$ ✓

Laufzeit? für Σ_1 gibt es $2^{|\Sigma_1|}$ mögliche Teilmengen, also $\leq 2^{\lfloor \frac{n}{2} \rfloor}$ Teilm.

für Σ_2 gibt es $2^{|\Sigma_2|}$ mögliche Teilmengen, also $\leq 2^{\lceil \frac{n}{2} \rceil}$ Teilm.

$\Rightarrow$ für beide Listen L_1 & L_2 gilt $|L_1|, |L_2| \in \mathcal{O}(2^{\frac{n}{2}})$

das berechnen einer einzelne möglichen Summe in L_1 & L_2 naiv in $\mathcal{O}(n)$ Zeit da es $\mathcal{O}(2^{\frac{n}{2}})$ solche Summen gibt die berechnet werden müssen kommen wir auf $\mathcal{O}(n 2^{\frac{n}{2}})$

Zu Bestimmung von L_1 bzw L_2

Hinweis: naiv alle Paare $x \in L_1$ & $y \in L_2$ zu testen würde in $\mathcal{O}(2^{\frac{n}{2}}) \cdot \mathcal{O}(2^{\frac{n}{2}}) = \mathcal{O}(2^n)$ gehen, dann haben wir also nix gewonnen im Vgl zu Brute Force.

der Trick ist für $x \in L_1$ müssen wir nur in L_2 schauen, ob $y = T - x$ existiert.

genauer. Sortiere L_2 in $O(|L_2| \cdot \log |L_2|)$ Zeit
da $|L_2| \leq 2^{\lceil \frac{n}{2} \rceil}$

$\Rightarrow$ Sortieren von L_2 geht in

$$O\left(2^{\frac{n}{2}} \cdot \log\left(2^{\frac{n}{2}}\right)\right) = O\left(2^{\frac{n}{2}} \cdot n\right) \text{ Zeit}$$

jetzt nimm $x \in L_1$ &
& check mittels Binärsuche ob
$y = T - x$ in L_2 existt.
$\Rightarrow$ Binärsuche in $O(\log_2(|L_2|)) =$
$O(\log_2(2^{\frac{n}{2}})) =$
$O(n)$ Zeit.

$\Rightarrow$ Subset-sum-DC kann so implementiert werden, dass die gesamt Laufzeit

$O(n \cdot 2^{\frac{n}{2}})$ ist [dafür allerdings Platz für hohen Speicherbedarf, da $O(2^{\frac{n}{2}})$ Zeug in L_1 & L_2 gespeichert werden müssen].

& $O(n \cdot 2^{\frac{n}{2}})$ ist schonmal ein Stück besser als $O(2^n)$

$n = 40$ $2^{40} \approx 1.1 \cdot 10^{12}$
$2^{20} \approx 1 \cdot 10^6$ das ist echter Gewinn an Laufzeit!

Frage: Wenn $\{1 \ldots n\}$ in 3 Teile I_1, I_2, I_3 zerlegt.
kommt man dann nicht auf $O(n 2^{\frac{n}{3}})$?

Antwort: Nein, man darf „kombinieren" schritt nicht vergessen!

Bei zwei Teilen suche $x + y = T \Rightarrow y = T - x$

Bei 3 Teilen suche $x + y + z$

↓ wenn x feststeht

kennen wir y & z nicht einzeln nur:

$$y + z = T - x$$

also müssen wir paare aus L_2 & L_3 kombinieren
was nun ergibt $2^{\frac{n}{3}} \cdot 2^{\frac{n}{3}} = 2^{\frac{2n}{3}} > 2^{\frac{n}{2}}$

also DIVIDE ist hier nützlich,
wenn der „kombinieren"-schritt günstig ist

3.2 Dynamische Programmierung

Dyn. Prg. ist vielleicht bekannt als Technik für Design von Poly. zeit alg.

IDEE Dyn. Prg: Zerlegen des Problems in überlappende Teilprobleme, diese Rekursiv lösen, dabei aber Zwischenlösungen speichern um wdh. Berechnungen zu vermeiden.

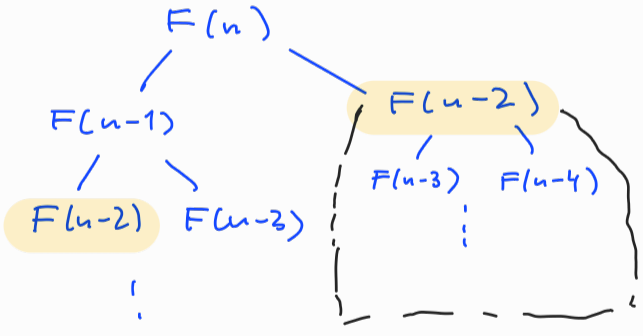
Bsp Fibonacci Nr $f(1) = f(2) = 1$

$f(n) = f(n-1) + f(n-2)$, $n > 2$

Naive Rekursive Weg:

"Problem dabei"

```
F(n)
  IF (n ≤ 2) THEN f = 1
  ELSE f = F(n-1) + F(n-2)
  Return f.
```



Laufzeit:

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

$$\geq 2\,T(n-2)$$

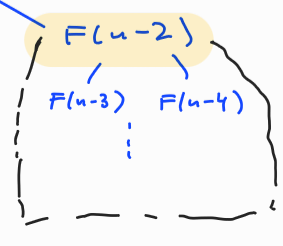
$T(n-1) \geq T(n-2)$

$$\geq 2 \cdot 2\,T(n-4)$$

$$\vdots$$

$$\geq 2^{n/2} \in \Omega(2^n).$$

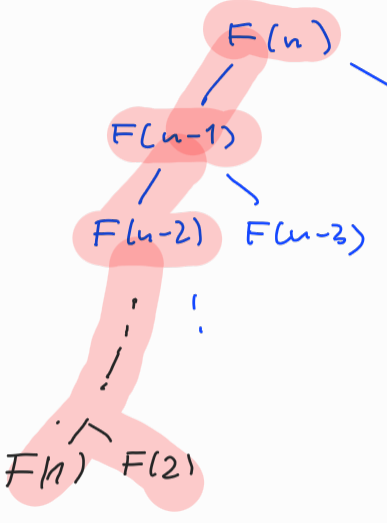
Berechnen $F(k)$ verschiedene male, obwohl wir es ja nur einmal machen müssten, als kämen dem gesamten Teilbaum



ignorieren.

```
memo = ∅   // dictionary
mF(n)
    IF (n ∈ memo) return memo[n]
    IF (n ≤ 2)  f = 1
    ELSE  f = mF(n-1) + mF(n-2)
    memo[n] = f
    return f
```

im wesentlichen brauchen wir nur wenn wir einen der vorherigen Pfade einmal berechnen & speichern, dann können wir hier, beim call von F(n-2) den wert und müssen nicht rekursiv wieder runter wandern



⇒ Laufzeit $O(n)$!

PROBLEM HAM_PFAD

Input: (un)gerichteter Graph $G = (V, E)$

Frage: Gibt es einen Hamilt. Pfad in G?

(Ham. Pfad enthält jeden Knoten genau einmal)

Brute Force: Sei $V = \{v_1 \dots v_n\}$

jeder Ham. Pfad in Index. ordnung

$v_{\pi(1)}\, v_{\pi(2)} \dots v_{\pi(n)}$ durchlaufen.

also könnte man alle Permutationen $\pi : \{1 \dots n\} \to \{1 \dots n\}$ aufzählen & für jedes π testen ob $v_{\pi(i)}\, v_{\pi(i+1)}$ eine Kante ist $\forall i \in \{1 \dots n-1\}$

Problem: es gibt $n!$ viele Permutationen, das ist sogar schlechter als 2^n:

$n \geq 4$, $2^n = \underbrace{2 \cdot 2 \cdot \ldots \cdot 2}_{n \text{ mal}} \leq 1 \cdot 2 \cdot 3 \cdot 4 \cdots n = n!$

$20! \approx 2.4 \cdot 10^{18}$ $\quad 2^{20} \approx 10^6$

$50! \approx 3 \cdot 10^{64}$ $\quad 2^{50} \approx 1{,}1 \cdot 10^{15}$

$\Rightarrow$ Laufzeit BruteForce $O(f(n) \cdot n!)$

$f(n)$ = Laufzeit für Test ob $v_{i_1}, v_{i_2}, \ldots, v_{i_n}$ Kante $\forall i$ in $O(n)$ ist wenn bspw. konst. time Zugriff auf Adj ist kante JA/NEIN.

da wir NP hartes Problem HAM-Pfad exakt lösen wollen können wir expon. Laufzeit im allg. nicht vermeiden.
aber $O(n!)$ ist zu schlecht!

Ist es möglich wie bei der Fib.nr., unnötige Berechnungen zu vermeiden?

Ann. wir haben Ham.Pfad Knoten für Knoten auf.
also bspw Teilpfad $s \to a \to b \to c$
oder anderer Teilpfad $s \to b \to a \to c$

wenn wir diese Pfade vervollständigen wollen, müssen wir in diesen Fällen

$$s \to a \to b \to c$$
$$s \to b \to a \to c$$

Ja nur wissen, dass wir s, a, b, c schon besucht haben & in beiden Fällen ist der Endpunkt von dem wir weiter machen der Knoten c.

$\Rightarrow$ müssen $S = \{s, a, b, c\}$ schon besucht & c kennen

$\Rightarrow$ alle weiteren Knoten sind in $V \setminus S$

um um den Pfad zu erweitern brauchen wir dann cv für ein $v \in V \setminus S$,

Für das "Folge"problem ist es irrelevant ob vorher $s \to a \to b \to c$ o. $s \to b \to a \to c$

gewählt wurde, d.h. die Folgenden Teilprobleme sind gleich.

Dyn. Prg. versucht nun das mehrfache Berechnen solcher Teilprobleme zu vermeiden.

Welche Inform. benötigen wir?

$S \subseteq V$ Menge der schon besuchten Knoten

+ Endknoten v des aktuellen Pfades, damit wir wissen wo wir weiter machen müssen

$\Rightarrow$ (S, v) mit $S \subseteq V$ & $v \in S$ //

Def $DP(S,v) \in \{TRUE, FALSE\}$

wie

$DP(S,v) = TRUE \iff$ es gibt einfachen Pfad in G od

- genau die Knoten in S besucht werden
- jeder Knoten in S wird genau einmal besucht (= einfacher Pfad)
- der Pfad endet in v.

Base Case: $S = \{v\}$ Pfad nur aus einem Knoten, der existiert immer!

$\Rightarrow DP(\{v\}, v) = TRUE \quad \forall v \in V$

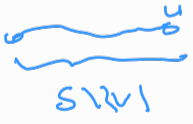
Betrachte jetzt $DP(S,v)$, $|S| \geq 2$.

da $|S| \geq 2$ & der Pfad in v endet, hat dieser Pfad eine Kante uv (u ist "Vorgänger" von v)

d.h. $u \in S \setminus v$ & $uv \in E$



$\Rightarrow$ wenn wir v löschen von diesem Pfad



bekommen wir einfachen Pfad der jeden Knoten aus $S \setminus \{v\}$ besucht & in u endet.

$\Rightarrow DP(S \setminus \{v\}, u) = TRUE$

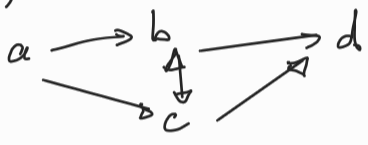
Umgekehrt, wenn $DP(S \setminus \{u\}, u) = \text{TRUE}$
& es hatte uv gibt

$\Rightarrow$ es gibt einfachen Pfad der alle Knoten in S besucht & in v endet

$\Rightarrow$ Erhalten Rekurrenz:

$$DP(S, v) = \bigvee_{\substack{u \in S \setminus \{v\} \\ uv \in E}}^{(OR)} DP(S \setminus \{v\}, u)$$

Bsp: $G = (V, E)$



wollen $DP(\{a,b,c,d\}, d)$ bestimmen.

mögliche Vorgänger von d: b & c

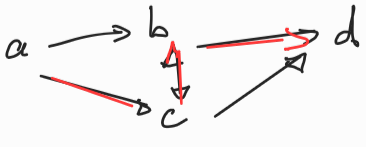
$\Rightarrow DP(\{a,b,c,d\}, d) = DP(\{a,b,c\}, b) \vee DP(\{a,b,c\}, c)$

TRUE TRUE

mögliche Vorgänger in $\{a,b,c\}$ von b sind a & c

$\Rightarrow DP(\{a,b,c\}, b) = DP(\{a,c\}, a) \vee DP(\{a,c\}, c)$

$DP(\{a,c\}, c) = DP(\{a\}, a) = \text{TRUE}$ TRUE



Rekurrenz $DP(S,v)$

benötigt ausschließlich Info's über $DP(S\setminus\{v\},u)$

size $|S|-1$

usw.

d.h. wir bestimmen erst alle Einträge für $|S|=1$

dann $|S|=2$

usw

$|S|=n$

FOR $\forall v \in V$: $DP(\{v\},v) = \text{TRUE}$

FOR $i = 2 \dots n$

FOR every $S \subseteq V$, $|S| = i$

$DP(S,v) = \text{FALSE}$

FOR every $u \in S \setminus \{v\}$

IF $uv \in E$ & $DP(S\setminus\{v\},u) = \text{TRUE}$

$DP(S,v) = \text{TRUE}$

BREAK.

$G=(V,E)$ enthält ham. Pfad

$\Leftrightarrow$ $\exists v \in V$ od $DP(V,v) = \text{TRUE}$.

$\Leftrightarrow$ $\bigvee_{v \in V} DP(V,v)$

Theorem: Für alle $S \subseteq V$ und alle $v \in S$ gilt

$DP(S,v) = \text{TRUE} \Leftrightarrow$ exist einfacher Pfad in G der alle Knoten aus S enthält & in v endet.

Beweis: Induktion über $|S|$.

Base case: $|S| = 1 \Rightarrow S = \{v\}$ für ein $v \in V$

Pfad existiert &

$DP(v, \{v\})$ wurde korrekt auf TRUE gesetzt.

Ann. Ind. Beh. gilt f.a. $S \subseteq V$ mit $|S| < r$, $r \geq 2$

Betrachte $S \subseteq V$, $|S| = r \geq 2$

Sei $v \in S$.

„$\Rightarrow$" Ann $DP(S, v) = TRUE$

* IF-Bed.

$\Rightarrow$ $\exists u \in S$

& $DP(S \setminus \{v\}, u) = TRUE$

da $|S \setminus \{v\}| = r - 1 \Rightarrow$ exist einfacher Pfad in G der alle Knoten aus $S \setminus \{v\}$ enthält & in u endet.

da $uv \in E$ können wir diesen Pfad entsprechend erweitern

$\Rightarrow$ exist einfacher Pfad in G der alle Knoten aus S enthält & in v endet.

„$\Leftarrow$" **Ann:** exist einfacher Pfad in G der alle Knoten aus S enthält & in v endet.

Da $|S| \geq 2 \Rightarrow$ es gibt Kante $(uv) \in E$,

d.h. $u \in S \setminus \{v\}$

Wenn wir v von diesem Pfad löschen bekommen wir einfach Pfad der alle Knoten in $S \setminus \{v\}$ enthält & in u endet.

da $|S \setminus \{v\}| < r \Rightarrow DP(S \setminus \{v\}, u) = \text{TRUE}$

$$\Rightarrow DP(S, v) = \bigvee_{\substack{u \in S \setminus \{v\} \\ uv \in E}}^{(OR)} DP(S \setminus \{v\}, u)$$

$$= \text{TRUE}$$

(also kommt in Alg auf TRUE gesetzt).

Laufzeit?

im wesentlichen betrachten wir alle $S \subseteq V$

$\Rightarrow 2^n$ mögliche subsets + Test ob Kante

$\Rightarrow \mathcal{O}(n \cdot 2^n)$ was deutlich besser als $O(n!)$ mit Brute Force ist

Anm: Dieselbe Technik kann genutzt werden um das TSP in $O(2^n n^2)$ exakt zu lösen (Held-Karp-Alg).

Unterschied: Ham. Pfad $D(S,v) \in \{\text{TRUE}, \text{FALSE}\}$, "Vor"

TSP $D(S,v) \in \mathbb{R}_{\geq 0}$, "min"

Zusf.:

Dyn. programming ist bekannt zur Konstruktion von Polyn. zeit Alg, also DP impliziert nicht notwendigerweise poly zeit & kann zum Lösen von NP-schweren Problem genutzt werden.

3.3. Suchräume einschränken (Branching)

Vertex-Cover Problem (VCP)

input: undirected graph $G = (V, E)$, integer L

question: Does G have a vertex cover of size $k \leq L$, i.e.,
is there a subset $C \subseteq V$
s.t.: (i) $|C| = k \leq L$
(ii) $C \cap e \neq \emptyset \quad \forall e \in E$

Brute Force: $\forall$ Teilmengen $C \subseteq V$,
prüfe ob C VC von G ist
& ob $|C| \leq k$

Laufzeit? $\mathcal{P}(V) = 2^n$, $|V| = n$, $|E| = m$
Test ob VC $\mathcal{O}(n+m)$

$\Rightarrow \quad \mathcal{O}(2^n (m+n))$.

Wir können die Laufzeit verbessern, indem wir Problem-spezifische Betrachtungen und einfügen lassen.

Bspw.: Im VC brauchen wir nur Subsets $C \subseteq V$ der Größe k zu testen!

Brute force mit exhaustive search.

| $\forall$ k-elementige Teilmenge $C \subseteq V$
| prüfe ob C VC von G ist

Laufzeit: es gibt $\binom{n}{k} = \frac{n!}{k!(n-k)!}$

$$= \frac{1 \cdot 2 \cdots (n-k)(n-k+1) \cdots n}{1 \cdot 2 \cdots (n-k) \cdot k!}$$

$$= \frac{(n-k+1) \cdots n}{k!} \leq n^k \in O(n^k)$$

$\Rightarrow O(n^k (m+n))$

schon besser zumindest für "kleine" k.

Aber immer noch, leichte Alg. nimmt noch keine schwierige Eigenschaft von G an.

Beob: Für jede kante $uv \in E$: $u \in C$ oder $v \in C$

strukt. Eigenschaft von G

Simple Technik: Suchraum eingrenzen:

Betrachte VC-problem $(G = (V, E), k)$

$\forall uv \in E$: $u \in C$ oder $v \in C$

aber mind. eines von u oder v muß in C liegen.

wenn $u \in C \Rightarrow$ genügt es $(G - u, k-1)$ zu betrachten, wobei

$G - u := (V - \{u\}, E - \{uw \mid uw \in E\})$

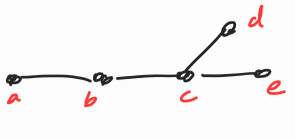
wenn $v \in C \Rightarrow$ betrachte $(G - v, k-1)$

mind. eines der Fälle muß auftreten.

```
VC_1(b,k)
    wenn E(b) = ∅   Return TRUE
    wenn k ≤ 0      Return FALSE
    wähle beliebige Kante (uv) ∈ E(G)
        Return VC_1(b-u, k-1) OR VC_1(G-v, k-1)
```

log. OR

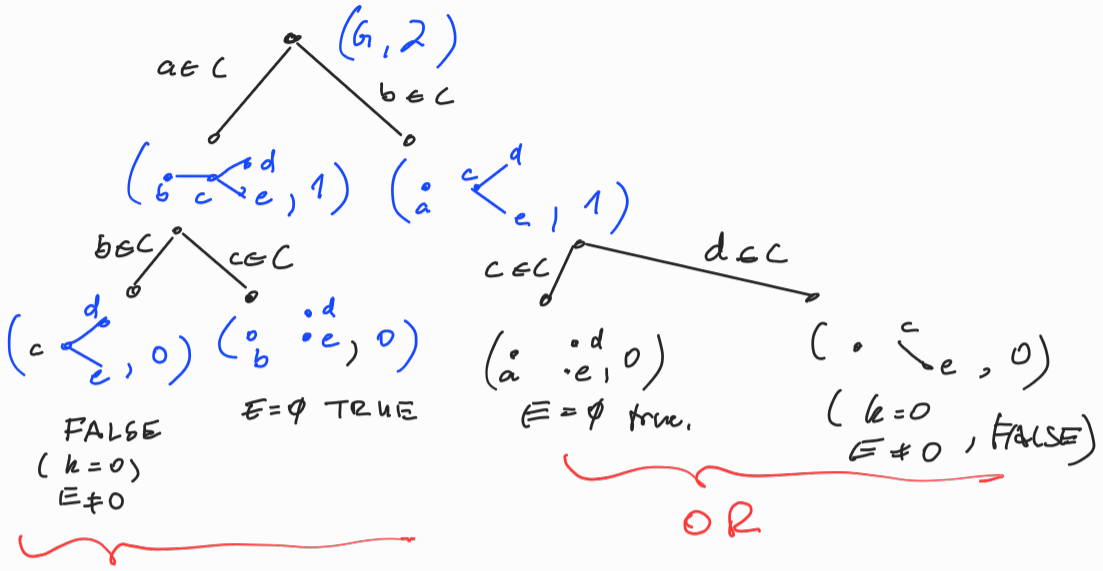
gibt uns ja/nein-Antwort.



$C = \{b, c\}$ opt VC. , $k = 2$

aber auch $C = \{a, c\}$

start obdA mit kante a — b.



• Korrektheit , da sind eine der Fälle aufrecht muß, dh.

(G, k) ist JA-Inst. $\Leftrightarrow$ $(G - u, k-1)$ oder $(G - v, k-1)$ ist JA-Inst. $\forall uv \in E(G)$.

$\Rightarrow$ VC_1 returned TRUE $\Leftrightarrow$ (G, k) ist JA-Inst. von VC.

(Üb. Aufg pro. Ind $E(G) = \emptyset$
$E(G) \neq \emptyset$ ..)

Laufzeit: Suchbaum (G,k) — 1 Knoten

$(G-u,k-1)$ $(G-v,k-1)$ — 2 Knoten

usw. — 4 Knoten

⋮

$(G-\dots,0)$ — 2^k Knoten

wenn $k=0$ ⇒ stop.

⇒ • Höhe Suchbaum ist k

• jeder nicht-Blatt Knoten hat mind 2 Kinder

⇒ **# Knoten** im Suchbaum ist

$$\sum_{i=1}^{k} 2^i = 2^{k+1} - 1 \in O(2^k).$$

Zur Laufzeitberechnung muss man noch den Overhead zum Test $E(G) = \emptyset$ & konstante & Kombinatorik um u finden $G-u$, $G-v$ dazurechnen.

das geht in $O(n+m)$ Zeit für jeden einzelnen Schritt (Übung)

⇒ **Laufzeit VC-1 ist $O(2^k(n+m))$.**

[Alternativ $T(k)$ = Laufzeit für bel. großen G

⇒ $T(k) \le T(k-1) + T(k-1) + O(n+m)$

$= 2T(k-1) + O(n+m)$ (und $G-u$ hat höchstens n Knoten & m Kanten)

$\le 2\left(T(k-2) + T(k-2) + O(n+m)\right) + O(n+m)$

$= 4T(k-2) + 3O(n+m)$

⋮

(nach i Schritten) $= 2^i T(k-i) + (2^i-1)O(n+m)$

⋮

$= 2^k \underbrace{T(0)}_{\text{konst. } O(1)} + (2^k-1)O(n+m) = O(2^k O(n+m))$]

Zusammenfassung:

Vergleich "naive" Ansatz vs VC_1

Laufzeit $\mathcal{O}(2^n (n+m))$ vs $\mathcal{O}(2^k (n+m))$.

↗ Hier haben wir die exponentielle Laufzeit auf den Parameter k geschoben. Für "kleine" k schnell lösbar!

Was nutzen wir?	Suchraumgröße	Bsp $n=100, k=5$
Lsg ist Teilmenge von V:	2^n	$2^{100} = 1.27 \cdot 10^{30}$
Lsg hat höchstens k Knoten:	$\binom{n}{k} \leq n^k$	$\approx 7.5 \cdot 10^7$
jeder Knoten erzwingt Entscheidung:	2^k $(k \leq n)$	32

Das Prinzip der Verzweigung (Branching) wie oben beim Vertex Cover Problem.

(G,k)

$(G-u,k-1)$ $(G-v,k-1)$

ist ein allg. algor. Prinzip.

- Finde lokale Strukturen die eine Lsg erfüllen muß
- Zerlege Lsg in kleine mögliche Fälle
- Löse die Fälle rekursiv

Zusf. allg:

NP-schwer $\not\Rightarrow$ "praktisch unlösbar"

Frage daher: Wie viel besser als Durchprobieren können wir werden?

Bisherige Techniken

D & C: Eingabe in unabhängige Teilprobleme zerlegen lösen & kombinieren

DP: überlappende Teilprobleme nur einmal lösen & Ergebnisse speichern

Welche Information aus der "Vergangenheit" hilft zukünftige Inform. zu bestimmen?

Branching: Problem durch Fallunterscheidung in alternative Fälle zerlegen.

Welche Wahl muß jede Lsg erzwingen?
(mit Hilfe struktur. Eigenschaften)

	naiv/brute force	Better way	
Vertex cover	$O(n^k)$	$O(2^k \cdot \text{poly}(n))$	wie branching
Subset sum	$O(2^n)$	$O(2^{\frac{n}{2}} \cdot \text{poly}(n))$	D & C
Ham. Pfad	$O(n!)$	$O(2^n \text{poly}(n))$	DP

NP – schwer ≠ hoffnungslos!

Auch wenn wir keine poly-zt Alg finden können, können wir durch Suchraum einschränkung, geschickte Zerlegung & wiederverwendung von Teillösungen oft exakt expon. Alg schalten, die deutlich besser sind als naives Brute Force

$n!$ schneller als
2^n schneller als
2^n schneller als
1.1^n ..